JAVA ENTERPRISE QUESTION BANK

Very Easy Code • Sample Output • Where to Write • How to Run • Viva Questions & Answers

---

Included: all 77 questions from the uploaded EJ QB question bank: Servlets, JSP, JDBC, EJB and JPA.

This booklet uses beginner-friendly classroom examples. Each question has code, expected output, a short explanation, file location/run steps, and one viva question with its answer.

Important: Each answer is a separate mini-exercise. Do not paste every question into one project, because several examples reuse simple class names and URL paths.

Servlet/JSP examples use the Java EE 8 style (`javax.*`) and are intended for Apache Tomcat 9. EJB/JPA examples need a Java EE server such as Payara 5 or GlassFish 5; plain Tomcat does not provide an EJB container. Tomcat 9 implements Servlet 4.0 and JSP 2.3. [Official Tomcat references are listed at the end.]

Quick Start: Where to Write Code

1. Java + IDE. Install a JDK and Apache NetBeans. Create a Java Web project for Servlet/JSP exercises. Add the Tomcat 9 server in NetBeans.

2. Servlet files. Put Java servlet classes in `Source Packages / src/java`. The Java filename must match the public class name. The `@WebServlet("/path")` annotation gives the URL path.

3. HTML and JSP. Put HTML and JSP files in `Web Pages`. Run the HTML/JSP page or open the servlet path such as `http://localhost:8080/YourProject/calculator`.

4. Database examples. Install MySQL and the MySQL Connector/J driver. Create a database and a shared helper class `app.DB`. Edit the password to match your own MySQL setup.

5. EJB/JPA examples. Create a Java EE application and deploy it on Payara 5 or GlassFish 5. Create `META-INF/persistence.xml` and configure the data source named `jdbc/CollegeDS` or use the data source available in your training lab. A JPA entity maps Java objects to database rows. [Official JPA references are listed at the end.]

6. Namespace warning. These examples use `javax.servlet`, `javax.ejb` and `javax.persistence` for a Java EE 8-style learning setup. Newer Jakarta EE 9+ servers use `jakarta.*` package names, so do not mix the two styles in the same project. [Official namespace references are listed at the end.]

Shared MySQL helper for JDBC examples

```
-- Run once in MySQL
CREATE DATABASE college;
USE college;

-- File: src/java/app/DB.java
package app;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class DB {
    public static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/college", "root", "YOUR_PASSWORD");
    }
}
```

Change YOUR_PASSWORD to your MySQL password. Add MySQL Connector/J to the project Libraries. Create the table named in each database question before running it.

Useful starter tables for the database questions

```
USE college;
CREATE TABLE users (username VARCHAR(50) PRIMARY KEY, password VARCHAR(100), email VARCHAR(100), country VARCHAR(50));
CREATE TABLE customers (id INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(80), email VARCHAR(100), phone VARCHAR(30));
CREATE TABLE qa (id INT AUTO_INCREMENT PRIMARY KEY, question VARCHAR(255), answer VARCHAR(500));
CREATE TABLE emp (empno INT PRIMARY KEY, ename VARCHAR(80), salary DOUBLE);
ALTER TABLE emp ADD age INT;
ALTER TABLE emp ADD designation VARCHAR(80);
CREATE TABLE gym (username VARCHAR(50) PRIMARY KEY, password VARCHAR(100));
```

Create only the tables needed by the question you are running. If a table already exists, do not run its CREATE TABLE statement again.

Shared note for EJB/JPA examples

```
<!-- Place in META-INF/persistence.xml of the Java EE application -->
<?xml version="1.0" encoding="UTF-8"?>
<persistence xmlns="http://xmlns.jcp.org/xml/ns/persistence" version="2.2">
  <persistence-unit name="AppPU" transaction-type="JTA">
    <jta-data-source>jdbc/CollegeDS</jta-data-source>
    <properties>
      <property name="javax.persistence.schema-generation.database.action" value="create"/>
    </properties>
  </persistence-unit>
</persistence>
```

`jdbc/CollegeDS` must be configured on Payara/GlassFish to point to your database. The schema-generation setting is suitable for a new practice database; use `none` or `update` only according to your server/provider and class instructions. Do not use `create` on a database with important data.

Contents

- Q1-A | SERVLETS - 26 questions
- Q1-B | JSP, JDBC AND WEB FORMS - 21 questions
- Q2-A | EJB APPLICATIONS - 12 questions
- Q2-B | JPA APPLICATIONS - 18 questions

Total: 77 questions with code, output, short explanation, run instructions and viva.

Q1-A | SERVLETS

Q1-A - Question 1: Create a simple calculator application using a Servlet.

Technology: Servlet

Where to write: src/java/CalculatorServlet.java

Easy explanation: The servlet reads two numbers and an operation, calculates the answer and prints it.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/calculator")
public class CalculatorServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>First number: <input type='number' name='a' required></label><br>");
        out.println("<label>Second number: <input type='number' name='b' required></label><br>");
        out.println("<label>Operation (+, -, *, /): <input name='op' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        double a = Double.parseDouble(request.getParameter("a"));
        double b = Double.parseDouble(request.getParameter("b"));
        String op = request.getParameter("op");
        double ans;
        switch (op) {
            case "+": ans = a + b; break;
            case "-": ans = a - b; break;
            case "*": ans = a * b; break;
            case "/":
                if (b == 0) { out.println("Cannot divide by zero"); return; }
                ans = a / b; break;
            default: out.println("Use +, -, * or /"); return;
        }
        out.println("Answer = " + ans);
    }
}
```

Sample output: For 10, 5 and *: `Answer = 50.0`

How to run: Run `/calculator`, enter two numbers and an operator such as `+`, then press Submit.

Viva question: What is a Servlet?

Viva answer: A Java class that handles web requests and sends responses.

Q1-A - Question 2: Develop a basic calculator using ServletInputStream and ServletOutputStream to perform addition, subtraction, multiplication, division and modulus.

Technology: Servlet streams

Where to write: src/java/StreamCalculatorServlet.java

Easy explanation: ServletInputStream reads the raw request body. ServletOutputStream writes the result. This example uses a simple comma-separated body.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;

@WebServlet("/streamcalc")
public class StreamCalculatorServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws IOException {
        // Send raw text as: 12,5,mod
        ServletInputStream in = req.getInputStream();
        ByteArrayOutputStream buffer = new ByteArrayOutputStream();
        int ch;
        while ((ch = in.read()) != -1) buffer.write(ch);
        String[] p = buffer.toString("UTF-8").trim().split(",");
        double a = Double.parseDouble(p[0]);
        double b = Double.parseDouble(p[1]);
        String op = p[2];
        double r;
        switch (op) {
            case "add": r = a + b; break;
            case "sub": r = a - b; break;
            case "mul": r = a * b; break;
            case "div": r = a / b; break;
            case "mod": r = a % b; break;
            default: r = 0;
        }
        res.setContentType("text/plain;charset=UTF-8");
        ServletOutputStream out = res.getOutputStream();
        out.print("Result = " + r);
    }
}
```

Sample output: Request body `12,5,mod` -> response `Result = 2.0`.

How to run: Deploy the servlet. Send a POST request to `/streamcalc` with plain-text body `12,5,add` (or sub/mul/div/mod). A REST client or curl can send the raw body.

Viva question: What is ServletInputStream used for?

Viva answer: It reads binary or raw data from the client request.

Q1-A - Question 3: Create a Servlet-based login page that displays “Hello” or “Hello [username]” for valid credentials and “login failed” for invalid credentials.

Technology: Servlet

Where to write: src/java/LoginServlet.java

Easy explanation: This beginner example checks a fixed username and password. Real applications should use a database and hashed passwords.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/login")
public class LoginServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Username: <input type='text' name='username' required></label><br>");
        out.println("<label>Password: <input type='password' name='password' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
```

```
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        String u = request.getParameter("username");
        String p = request.getParameter("password");
        if ("student".equals(u) && "1234".equals(p))
            out.println("Hello " + u);
        else
            out.println("login failed");
    }
}
```

Sample output: Valid: `Hello student`
Invalid: `login failed`

How to run: Open `/login`. Try username `student` and password `1234`.

Viva question: What is authentication?

Viva answer: Checking whether a user is allowed to log in.

Q1-A - Question 4: Create a Servlet-based login application for an online examination portal.

Technology: Servlet + Session

Where to write: src/java/ExamLoginServlet.java

Easy explanation: A simple login for students. A session remembers the student after successful login.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/examlogin")
public class ExamLoginServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Student name: <input type='text' name='name' required></label><br>");
        out.println("<label>Password: <input type='password' name='password' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        String name = request.getParameter("name");
        String password = request.getParameter("password");
        if ("exam123".equals(password)) {
            request.getSession().setAttribute("student", name);
            out.println("Welcome " + name + " to the Online Examination Portal");
            out.println("<br><a href='exam'>Start Exam</a>");
        } else out.println("Login failed");
    }
}
```

Sample output: Welcome Riya to the Online Examination Portal
Start Exam

How to run: Open `/examlogin`; use any name and password `exam123`.

Viva question: Why is a session useful in an exam portal?

Viva answer: It remembers the logged-in student between pages.

Q1-A - Question 5: Create a registration Servlet using JDBC to accept username, password, email and country from an HTML form and store the details in a database.

Technology: Servlet + JDBC

Where to write: src/java/RegisterServlet.java; shared `app/DB.java`; MySQL table `users`

Easy explanation: The form values are stored using a prepared SQL statement. Create the table first and configure the shared DB helper shown in the setup section.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/register")
public class RegisterServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Username: <input type='text' name='username' required></label><br>");
        out.println("<label>Password: <input type='password' name='password' required></label><br>");
        out.println("<label>Email: <input type='email' name='email' required></label><br>");
        out.println("<label>Country: <input type='text' name='country' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try (java.sql.Connection con = app.DB.getConnection();
             java.sql.PreparedStatement ps = con.prepareStatement(
                 "INSERT INTO users(username,password,email,country) VALUES(?,?,?,?)")) {
            ps.setString(1, request.getParameter("username"));
            ps.setString(2, request.getParameter("password"));
            ps.setString(3, request.getParameter("email"));
            ps.setString(4, request.getParameter("country"));
            ps.executeUpdate();
            out.println("Registration saved");
        } catch (Exception e) { out.println("Database error: " + e.getMessage()); }
    }
}
```

Sample output: Registration saved

How to run: Run `/register`; submit the form after creating the `users` table.

Viva question: Why use PreparedStatement?

Viva answer: It safely passes values to SQL and helps prevent SQL injection.

Q1-A - Question 6: Create a Servlet using RequestDispatcher to validate a password and forward the user to a Welcome Servlet when valid; otherwise, display an error or remain on the index page.

Technology: Servlet + RequestDispatcher

Where to write: Create `PasswordServlet.java` and `WelcomeServlet.java` as two separate files; form can be `index.html`.

Easy explanation: RequestDispatcher forwards a valid request to another servlet without making a new browser request.

Code

```
// File: PasswordServlet.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/checkpassword")
public class PasswordServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        if ("hello123".equals(req.getParameter("password"))) {
            req.setAttribute("name", req.getParameter("name"));
            req.getRequestDispatcher("/welcome").forward(req, res);
        } else {
            res.setContentType("text/html");
            res.getWriter().println("Wrong password. <a href='index.html'>Try again</a>");
        }
    }
```

```
}
// File: WelcomeServlet.java
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/welcome")
public class WelcomeServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.getWriter().println("Welcome " + req.getAttribute("name"));
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
        doGet(req, res);
    }
}
```

Sample output: Valid password: Welcome Sam
Wrong password: Wrong password. Try again

How to run: Create an HTML form that posts name/password to `/checkpassword`; valid password is `hello123`.

Viva question: What does forward() do?

Viva answer: It passes the same request and response to another server resource.

Q1-A - Question 7: Create a Servlet that uses cookies to store and display the number of times a user has visited the application.

Technology: Servlet + Cookies

Where to write: src/java/VisitCookieServlet.java

Easy explanation: A cookie stores a small value in the browser. The next visit sends the cookie back to the servlet.

Code

```
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/visits")
public class VisitCookieServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        int count = 0;
        Cookie[] cookies = req.getCookies();
        if (cookies != null) {
            for (Cookie c : cookies) {
                if ("visits".equals(c.getName())) count = Integer.parseInt(c.getValue());
            }
        }
        count++;
        Cookie c = new Cookie("visits", String.valueOf(count));
        c.setMaxAge(7 * 24 * 60 * 60);
        res.addCookie(c);
        res.setContentType("text/html");
        res.getWriter().println("You visited " + count + " time(s).");
    }
}
```

Sample output: You visited 1 time(s).
Refresh: You visited 2 time(s).

How to run: Open `/visits` and refresh the page.

Viva question: Where is a cookie stored?

Viva answer: In the user's browser.

Q1-A - Question 8: Create a Servlet that demonstrates session creation and destruction and checks whether the user is visiting for the first time or has visited previously.

Technology: Servlet + Session

Where to write: src/java/SessionDemoServlet.java

Easy explanation: HttpSession stores values while the user's session exists. `invalidate()` ends the session.

Code

```
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/sessiondemo")
public class SessionDemoServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        HttpSession session = req.getSession();
        Integer n = (Integer) session.getAttribute("count");
        if (n == null) n = 0;
        n++;
        session.setAttribute("count", n);
        PrintWriter out = res.getWriter();
        out.println(n == 1 ? "First visit in this session" : "Welcome back in this session");
        out.println("<br>Visit count: " + n);
        out.println("<br><a href='sessiondemo?end=yes'>Destroy session</a>");
        if ("yes".equals(req.getParameter("end"))) {
            session.invalidate();
            out.println("<br>Session destroyed. Refresh to start a new session.");
        }
    }
}
```

Sample output: First visit in this session
Visit count: 1
Next refresh: Welcome back in this session

How to run: Open `/sessiondemo`, refresh, and then click Destroy session.

Viva question: How do you destroy a session?

Viva answer: Call `session.invalidate()`.

Q1-A - Question 9: Create a Servlet application to upload and download files.

Technology: Servlet file upload/download

Where to write: Create `UploadServlet.java` and `DownloadServlet.java` separately. Files are saved in a folder named `servlet_uploads` inside your home folder.

Easy explanation: `@MultipartConfig` allows the servlet to receive uploaded files. The second servlet streams a saved file back to the browser.

Code

```
// File: UploadServlet.java
import java.io.*;
import javax.servlet.annotation.*;
import javax.servlet.http.*;
@WebServlet("/upload")
@MultipartConfig
public class UploadServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        res.getWriter().println("<form method='post' enctype='multipart/form-data'>"
            + "<input type='file' name='file' required><button>Upload</button></form>");
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException, javax.servlet.ServletException {
        Part part = req.getPart("file");
        String name = new File(part.getSubmittedFileName()).getName();
        File folder = new File(System.getProperty("user.home"), "servlet_uploads");
        folder.mkdirs();
        part.write(new File(folder, name).getAbsolutePath());
        res.getWriter().println("Saved file: " + name + " in " + folder.getAbsolutePath());
    }
}
// File: DownloadServlet.java
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/download")
public class DownloadServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        String name = new File(req.getParameter("name")).getName();
```

```
        File file = new File(new File(System.getProperty("user.home"), "servlet_uploads"), name);
        if (!file.isFile()) { res.getWriter().println("File not found"); return; }
        res.setHeader("Content-Disposition", "attachment; filename="" + name + """);
        try (InputStream in = new FileInputStream(file); OutputStream out = res.getOutputStream()) {
            byte[] b = new byte[4096]; int n;
            while ((n = in.read(b)) > 0) out.write(b, 0, n);
        }
    }
}
```

Sample output: Upload response: Saved file: notes.txt in .../servlet_uploads
Download URL: /download?name=notes.txt

How to run: Open `/upload` and select a file. For downloading, open `/download?name=your-file-name`. Use only trusted files.

Viva question: Why is @MultipartConfig used?

Viva answer: It enables multipart form-data file uploads.

Q1-A - Question 10: Create a Servlet application demonstrating non-blocking read operations.

Technology: Servlet 3.1 non-blocking I/O

Where to write: src/java/AsyncReadServlet.java

Easy explanation: The servlet registers a ReadListener and reads characters only when the input stream is ready.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet(value="/asyncread", asyncSupported=true)
public class AsyncReadServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res)
            throws IOException {
        AsyncContext ctx = req.startAsync();
        ServletInputStream in = req.getInputStream();
        StringBuilder data = new StringBuilder();
        in.setReadListener(new ReadListener() {
            public void onDataAvailable() throws IOException {
                while (in.isReady() && !in.isFinished()) {
                    int ch = in.read();
                    if (ch == -1) break;
                    data.append((char) ch);
                }
            }
            public void onAllDataRead() throws IOException {
                res.setContentType("text/plain;charset=UTF-8");
                res.getWriter().println("Read body: " + data);
                ctx.complete();
            }
            public void onError(Throwable t) { ctx.complete(); }
        });
    }
}
```

Sample output: For body hello, output is: Read body: hello.

How to run: Deploy and POST any small body to `/asyncread`.

Viva question: What is non-blocking I/O?

Viva answer: It lets a program continue other work instead of waiting for input to be ready.

Q1-A - Question 11: Develop a Servlet-based question-and-answer application that retrieves questions from a database.

Technology: Servlet + JDBC

Where to write: src/java/QuestionAnswerServlet.java; table `qa(question, answer)`

Easy explanation: The servlet runs a SELECT query and displays each question-answer row. Add rows to the `qa` table first.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/qa")
public class QuestionAnswerServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<p>This page reads all questions and answers from the database.</p>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try (java.sql.Connection con = app.DB.getConnection();
             java.sql.Statement st = con.createStatement();
             java.sql.ResultSet rs = st.executeQuery("SELECT question, answer FROM qa")) {
            while (rs.next()) {
                out.println("<p><b>Q:</b> " + rs.getString("question") + "<br><b>A:</b> "
                            + rs.getString("answer") + "</p>");
            }
        } catch (Exception e) { out.println("Database error: " + e.getMessage()); }
    }
}
```

Sample output: Q: What is Java?
A: A programming language.

How to run: Create the `qa` table, insert a few rows, start Tomcat and open `/qa`.

Viva question: What does ResultSet do?

Viva answer: It holds the rows returned by a SQL query.

Q1-A - Question 12: Develop a web-based multiple-choice quiz with at least five questions, four options per question and one selectable answer per question. Display the number of correct and incorrect answers after submission.

Technology: Servlet

Where to write: src/java/QuizServlet.java

Easy explanation: The servlet shows five questions and four options for each. Each question uses radio buttons so one answer can be selected.

Code

```
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/quiz")
public class QuizServlet extends HttpServlet {
    String[] questions = {"2 + 2 = ?", "Capital of India?", "Java is a...?", "HTML means?", "Red + blue makes?"};
    String[][] opts = {{"3","4","5","6"},{"Mumbai","Delhi","Pune","Goa"},
        {"Language","Car","OS","Browser"},{"Markup language","Database","CPU","Network"},
        {"Green","Purple","Yellow","Black"}};
    String[] answer = {"B","B","A","A","B"};
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html"); PrintWriter out = res.getWriter();
        out.println("<form method='post'>");
        for (int i=0; i<questions.length; i++) {
            out.println("<p>"+(i+1)+". "+questions[i]+"</p>");
            for (int j=0; j<4; j++) {
                char letter = (char)('A'+j);
                out.println("<label><input type='radio' name='q"+i+"' value='"+letter+"' required> "
                    +letter+". "+opts[i][j]+"</label><br>");
            }
```

```
        }
        out.println("<button>Submit quiz</button></form>");
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
        int correct=0;
        for (int i=0; i<answer.length; i++) if (answer[i].equals(req.getParameter("q"+i))) correct++;
        res.getWriter().println("Correct: "+correct+"<br>Incorrect: "+(answer.length-correct));
    }
}
```

Sample output: Correct: 4
Incorrect: 1

How to run: Open `/quiz`, answer all five questions, and submit.

Viva question: Why use radio buttons?

Viva answer: They let the user select one answer from a group.

Q1-A - Question 13: Create a Servlet application that accepts two numbers and an arithmetic operation from an HTML form, calculates the result and displays it.

Technology: Servlet

Where to write: src/java/OperationServlet.java

Easy explanation: The form sends two numbers and an operation name. The servlet selects the calculation.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/operation")
public class OperationServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Number 1: <input type='number' name='a' required></label><br>");
        out.println("<label>Number 2: <input type='number' name='b' required></label><br>");
        out.println("<label>Operation: add, sub, mul, div: <input type='text' name='op' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        double a = Double.parseDouble(request.getParameter("a"));
        double b = Double.parseDouble(request.getParameter("b"));
        String op = request.getParameter("op");
        double r;
        if ("add".equals(op)) r = a + b;
        else if ("sub".equals(op)) r = a - b;
        else if ("mul".equals(op)) r = a * b;
        else if ("div".equals(op) && b != 0) r = a / b;
        else { out.println("Invalid operation or division by zero"); return; }
        out.println("Result = " + r);
    }
}
```

Sample output: Number 1: 8, Number 2: 2, op: div -> Result = 4.0

How to run: Open `/operation`, enter values and use `add`, `sub`, `mul` or `div`.

Viva question: What does doPost() do?

Viva answer: It handles HTTP POST form submissions.

Q1-A - Question 14: Create a Servlet application demonstrating simple input validation.

Technology: Servlet validation

Where to write: src/java/ValidationServlet.java

Easy explanation: Validation checks that the name is not empty and that age is in a sensible range.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/validate")
public class ValidationServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Name: <input type='text' name='name' required></label><br>");
        out.println("<label>Age: <input type='number' name='age' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        String name = request.getParameter("name").trim();
        String ageText = request.getParameter("age");
        if (name.isEmpty()) { out.println("Name is required"); return; }
        try {
            int age = Integer.parseInt(ageText);
            if (age < 1 || age > 120) out.println("Enter a valid age");
            else out.println("Input accepted for " + name);
        } catch (NumberFormatException e) { out.println("Age must be a number"); }
    }
}
```

Sample output: Input: name=Riya, age=14 -> Input accepted for Riya

How to run: Open `/validate` and try valid and invalid values.

Viva question: What is input validation?

Viva answer: Checking input before using or saving it.

Q1-A - Question 15: Create a simple HTML form that submits a user's name to a Servlet and displays a welcome message containing the name.

Technology: HTML + Servlet

Where to write: `Web Pages/name.html` and `src/java/WelcomeNameServlet.java`

Easy explanation: The HTML form sends the name to the servlet. The servlet reads the parameter and displays it.

Code

```
<!-- File: name.html in Web Pages -->
<form action="welcomeName" method="post">
  Name: <input name="name" required>
  <button type="submit">Send</button>
</form>

// File: WelcomeNameServlet.java
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/welcomeName")
public class WelcomeNameServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html;charset=UTF-8");
        res.getWriter().println("Welcome " + req.getParameter("name"));
    }
}
```

Sample output: Welcome Riya

How to run: Run `name.html`, type a name and press Send.

Viva question: How does a servlet read a form field?

Viva answer: Use `request.getParameter("fieldName")`.

Q1-A - Question 16: Create a Servlet application for a stationery shop that accepts the prices of two items and displays their sum, difference, product and division.

Technology: Servlet

Where to write: src/java/StationeryServlet.java

Easy explanation: It reads two item prices and shows four arithmetic results.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/stationery")
public class StationeryServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Pen price: <input type='number' name='a' required></label><br>");
        out.println("<label>Notebook price: <input type='number' name='b' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        double a = Double.parseDouble(request.getParameter("a"));
        double b = Double.parseDouble(request.getParameter("b"));
        out.println("Sum = " + (a+b) + "<br>");
        out.println("Difference = " + (a-b) + "<br>");
        out.println("Product = " + (a*b) + "<br>");
        if (b != 0) out.println("Division = " + (a/b));
        else out.println("Division not possible: second price is zero");
    }
}
```

Sample output: Prices 20 and 5 -> Sum 25, Difference 15, Product 100, Division 4.0

How to run: Open `/stationery` and enter two prices.

Viva question: What is the modulus operator?

Viva answer: It gives the remainder after division, written `%`.

Q1-A - Question 17: Create a Servlet application for a canteen that accepts the prices of two food items and displays their total cost.

Technology: Servlet

Where to write: src/java/CanteenServlet.java

Easy explanation: The total cost is the sum of the two food prices.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/canteen")
public class CanteenServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Food item 1 price: <input type='number' name='p1' required></label><br>");
        out.println("<label>Food item 2 price: <input type='number' name='p2' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        double p1 = Double.parseDouble(request.getParameter("p1"));
        double p2 = Double.parseDouble(request.getParameter("p2"));
```

```
        out.println("Total cost = Rs. " + (p1+p2));
    }
}
```

Sample output: Prices 35 and 40 -> Total cost = Rs. 75.0

How to run: Open `/canteen` and enter both prices.

Viva question: Which operator adds two values in Java?

Viva answer: The plus operator `+`.

Q1-A - Question 18: Create a Servlet application for a college admission portal that validates an application and uses RequestDispatcher to forward it to a confirmation page.

Technology: Servlet + RequestDispatcher

Where to write: `src/java/AdmissionServlet.java` and `Web Pages/admissionConfirm.jsp`

Easy explanation: The servlet checks marks. A valid application is forwarded to a JSP confirmation page.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/admission")
public class AdmissionServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        res.getWriter().println("<form method='post'>Name: <input name='name' required> "
            + "Marks: <input type='number' name='marks' required><button>Apply</button></form>");
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
        String name = req.getParameter("name");
        int marks = Integer.parseInt(req.getParameter("marks"));
        if (marks >= 35) {
            req.setAttribute("applicant", name);
            req.getRequestDispatcher("/admissionConfirm.jsp").forward(req, res);
        } else res.getWriter().println("Application not accepted: minimum 35 marks required");
    }
}
```

Sample output: admissionConfirm.jsp: Application received for Riya

How to run: Create `admissionConfirm.jsp` containing `<h2>Application received for ${requestScope.applicant}</h2>`. Open `/admission`.

Viva question: What is a request attribute?

Viva answer: A value stored on the request so another resource can read it.

Q1-A - Question 19: Create a Servlet application for a bookshop that accepts customer details and stores them in a database using JDBC.

Technology: Servlet + JDBC

Where to write: src/java/BookshopCustomerServlet.java; MySQL table `customers`

Easy explanation: Customer details are inserted into the database with a prepared statement.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/bookcustomer")
public class BookshopCustomerServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Name: <input type='text' name='name' required></label><br>");
        out.println("<label>Email: <input type='email' name='email' required></label><br>");
        out.println("<label>Phone: <input type='text' name='phone' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
```

```
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        try (java.sql.Connection con = app.DB.getConnection();
             java.sql.PreparedStatement ps = con.prepareStatement(
                 "INSERT INTO customers(name,email,phone) VALUES(?,?,?)")) {
            ps.setString(1, request.getParameter("name"));
            ps.setString(2, request.getParameter("email"));
            ps.setString(3, request.getParameter("phone"));
            ps.executeUpdate();
            out.println("Customer saved");
        } catch (Exception e) { out.println("Database error: " + e.getMessage()); }
    }
}
```

Sample output: Customer saved

How to run: Create the `customers` table, configure `app.DB`, then open `/bookcustomer`.

Viva question: What is JDBC?

Viva answer: Java Database Connectivity; it lets Java programs work with databases.

Q1-A - Question 20: Create a Servlet application for a parcel-tracking system that uses sessions to store and display customer tracking details.

Technology: Servlet + Session

Where to write: src/java/ParcelServlet.java

Easy explanation: Session attributes hold the customer name and tracking number while browsing the application.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/parcel")
public class ParcelServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Customer name: <input type='text' name='name' required></label><br>");
        out.println("<label>Tracking number: <input type='text' name='tracking' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        HttpSession s = request.getSession();
        s.setAttribute("customer", request.getParameter("name"));
        s.setAttribute("tracking", request.getParameter("tracking"));
        out.println("Customer: " + s.getAttribute("customer") + "<br>");
        out.println("Tracking number: " + s.getAttribute("tracking"));
    }
}
```

Sample output: Customer: Riya
Tracking number: PK12345

How to run: Open `/parcel`, enter customer and tracking details, then refresh to see the saved session values by adding display code as needed.

Viva question: How do you store data in a session?

Viva answer: Use `session.setAttribute(key, value)`.

Q1-A - Question 21: Create a Servlet-based online test portal that stores student details in sessions and displays them on another page.

Technology: Servlet + Session

Where to write: Create `TestStudentServlet.java` and `StudentDetailsServlet.java` separately.

Easy explanation: The first servlet stores details in the session and redirects to a second servlet to display them.

Code

```
// File: TestStudentServlet.java
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/studenttest")
public class TestStudentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
        HttpSession s = req.getSession();
        s.setAttribute("name", req.getParameter("name"));
        s.setAttribute("roll", req.getParameter("roll"));
        res.sendRedirect("studentdetails");
    }
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        res.getWriter().println("<form method='post'>Name: <input name='name'> Roll: <input name='roll'><button>Save</button></form>");
    }
}
// File: StudentDetailsServlet.java
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/studentdetails")
public class StudentDetailsServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        HttpSession s = req.getSession(false);
        if (s == null || s.getAttribute("name") == null) { res.getWriter().println("Enter details first"); return; }
        res.getWriter().println("Student: " + s.getAttribute("name") + " | Roll: " + s.getAttribute("roll"));
    }
}
```

Sample output: Student: Riya | Roll: 12

How to run: Open `/studenttest`, enter details and submit.

Viva question: What is sendRedirect()?

Viva answer: It asks the browser to make a new request to another URL.

Q1-A - Question 22: Create a Servlet application for a shopping website that uses cookies to remember a visitor's name on subsequent visits.

Technology: Servlet + Cookies

Where to write: src/java/RememberNameServlet.java

Easy explanation: The servlet saves the visitor name in a cookie and reads it again on the next visit.

Code

```
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/remembername")
public class RememberNameServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();
        String saved = null;
        Cookie[] cookies = req.getCookies();
        if (cookies != null) {
            for (Cookie c : cookies) {
                if ("visitorName".equals(c.getName())) saved = c.getValue();
            }
        }
        if (saved != null) out.println("Welcome back, " + saved + "!<br>");
        out.println("<form method='post'>Name: <input name='name' required><button>Save name</button></form>");
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException {
        String name = req.getParameter("name");
        Cookie c = new Cookie("visitorName", name);
        c.setMaxAge(30 * 24 * 60 * 60);
        res.addCookie(c);
        res.setContentType("text/html");
        res.getWriter().println("Hello " + name + ". Your name is saved.");
    }
```

```
}
```

Sample output: First submit: Hello Riya. Your name is saved. Next visit: Welcome back, Riya!

How to run: Open `/remembername`, submit a name, then revisit or refresh the URL.

Viva question: How long can a cookie live?

Viva answer: It can be given an expiry time using `setMaxAge()`; the browser may also delete it.

Q1-A - Question 23: Create a Servlet application that calculates a water bill based on units consumed.

Technology: Servlet

Where to write: src/java/WaterBillServlet.java

Easy explanation: The bill uses example slab rates. Replace these rates with the rate chart your teacher provides.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/waterbill")
public class WaterBillServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Units consumed: <input type='number' name='units' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        int u = Integer.parseInt(request.getParameter("units"));
        if (u < 0) { out.println("Units cannot be negative"); return; }
        double bill;
        if (u <= 100) bill = u * 2.0;
        else if (u <= 200) bill = 100*2.0 + (u-100)*3.0;
        else bill = 100*2.0 + 100*3.0 + (u-200)*5.0;
        out.println("Water bill = Rs. " + bill);
        out.println("<br>Demo rates: first 100 at Rs.2, next 100 at Rs.3, above 200 at Rs.5.");
    }
}
```

Sample output: Units 250 -> Water bill = Rs. 750.0

How to run: Open `/waterbill` and enter units.

Viva question: What is a slab rate?

Viva answer: A rate that changes when usage crosses a set limit.

Q1-A - Question 24: Create a Servlet application for a college notice board that allows an administrator to upload PDF notices and students to download them.

Technology: Servlet file upload/download

Where to write: Create both servlet files separately. Only permit trusted administrator uploads in a real application.

Easy explanation: The upload servlet saves a PDF in a folder; the download servlet streams it to the student. Production applications also need administrator login and safe file storage.

Code

```
// File: NoticeUploadServlet.java
import java.io.*;
import javax.servlet.annotation.*;
import javax.servlet.http.*;
@WebServlet("/noticeupload")
@MultipartConfig
public class NoticeUploadServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        res.getWriter().println("<form method='post' enctype='multipart/form-data'>"
            + "Select PDF: <input type='file' name='file' accept='application/pdf' required><button>Upload</button></form>");
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws IOException, javax.servlet.ServletException {
        Part p = req.getPart("file");
        String name = new File(p.getSubmittedFileName()).getName();
        if (!name.toLowerCase().endsWith(".pdf")) { res.getWriter().println("PDF files only"); return; }
        File dir = new File(System.getProperty("user.home"), "college_notices"); dir.mkdirs();
        p.write(new File(dir, name).getAbsolutePath());
        res.getWriter().println("Uploaded. Download: <a href='noticedownload?name="+name+"'>"+name+"</a>");
    }
}
// File: NoticeDownloadServlet.java
import java.io.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/noticedownload")
public class NoticeDownloadServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        String name = new File(req.getParameter("name")).getName();
        File f = new File(new File(System.getProperty("user.home"), "college_notices"), name);
        if (!f.isFile()) { res.getWriter().println("Notice not found"); return; }
        res.setContentType("application/pdf");
        res.setHeader("Content-Disposition", "attachment; filename=""+name+""");
        try (InputStream in=new FileInputStream(f); OutputStream out=res.getOutputStream()) {
            byte[] b=new byte[4096]; int n; while((n=in.read(b))>0) out.write(b,0,n);
        }
    }
}
```

Sample output: Uploaded. Download: notice.pdf

How to run: Open `/noticeupload`, upload a PDF, then click the generated download link.

Viva question: Why check the file extension?

Viva answer: To reject obviously wrong file types; stronger content validation is also needed in real systems.

Q1-A - Question 25: Create a Servlet application for a library that accepts book details and displays the submitted information on a separate page.

Technology: Servlet + JSP

Where to write: `src/java/BookEntryServlet.java` and `Web Pages/bookdetails.jsp`

Easy explanation: The servlet accepts book details and forwards them to a separate JSP page using request attributes.

Code

```
// File: BookEntryServlet.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/bookentry")
public class BookEntryServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        res.getWriter().println("<form method='post'>Title: <input name='title'> Author: <input name='author'><button>Submit</button></form
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
        req.setAttribute("title", req.getParameter("title"));
        req.setAttribute("author", req.getParameter("author"));
        req.getRequestDispatcher("/bookdetails.jsp").forward(req, res);
```

```
    }
}
```

Sample output: bookdetails.jsp displays: Book title: Java Basics; Author: A. Kumar

How to run: Create `bookdetails.jsp` with `${requestScope.title}` and `${requestScope.author}`. Open `/bookentry`.

Viva question: What is JSP?

Viva answer: JavaServer Pages lets you create dynamic web pages.

Q1-A - Question 26: Create a Servlet application for a college event that uses RequestDispatcher to forward a valid registration to a confirmation page.

Technology: Servlet + RequestDispatcher

Where to write: `src/java/EventRegisterServlet.java` and `Web Pages/eventconfirm.jsp`

Easy explanation: The servlet checks the name and email, then forwards valid details to a confirmation JSP.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/eventregister")
public class EventRegisterServlet extends HttpServlet {
    protected void doGet(HttpServletRequest req, HttpServletResponse res) throws IOException {
        res.setContentType("text/html");
        res.getWriter().println("<form method='post'>Name: <input name='name' required> "
            + "Email: <input type='email' name='email' required><button>Register</button></form>");
    }
    protected void doPost(HttpServletRequest req, HttpServletResponse res) throws ServletException, IOException {
        String name=req.getParameter("name"); String email=req.getParameter("email");
        if (name.trim().isEmpty() || !email.contains("@")) { res.getWriter().println("Please enter valid details"); return; }
        req.setAttribute("name", name); req.setAttribute("email", email);
        req.getRequestDispatcher("/eventconfirm.jsp").forward(req, res);
    }
}
```

Sample output: Registration confirmed for Riya (riya@example.com)

How to run: Create `eventconfirm.jsp` using `${requestScope.name}` and `${requestScope.email}`. Open `/eventregister`.

Viva question: What is the purpose of a confirmation page?

Viva answer: It tells the user that the form was received successfully.

Q1-B | JSP, JDBC AND WEB FORMS

Q1-B - Question 27: Develop a JSP application to display values obtained from different implicit objects.

Technology: JSP

Where to write: `Web Pages/implicit.jsp`

Easy explanation: JSP provides objects such as request, response, session, application, out and page without needing to declare them.

Code

```
<!-- File: implicit.jsp -->
<html><body>
<h2>JSP Implicit Objects</h2>
Request method: <%= request.getMethod() %><br>
Server: <%= application.getServerInfo() %><br>
Session ID: <%= session.getId() %><br>
Page name: <%= page.getClass().getName() %><br>
<% out.println("Printed using the out object"); %>
</body></html>
```

Sample output: Request method: GET
Server: Apache Tomcat...
Session ID: (generated value)
Printed using the out object

How to run: Run `implicit.jsp` directly from the web project.

Viva question: Name any two JSP implicit objects.

Viva answer: `request` and `session`.

Q1-B - Question 28: Create a JSP application demonstrating at least three methods of four implicit objects.

Technology: JSP

Where to write: `Web Pages/implicitMethods.jsp`

Easy explanation: This page demonstrates methods of four implicit objects: request, session, application and out.

Code

```
<!-- File: implicitMethods.jsp -->
<%@ page contentType="text/html;charset=UTF-8" %>
<h2>Methods of four implicit objects</h2>
Request method: <%= request.getMethod() %><br>
Request URI: <%= request.getRequestURI() %><br>
Server name: <%= request.getServerName() %><br>
Session ID: <%= session.getId() %><br>
Session creation time: <%= session.getCreationTime() %><br>
Last access time: <%= session.getLastAccessedTime() %><br>
App server: <%= application.getServerInfo() %><br>
App context: <%= application.getContextPath() %><br>
Context name: <%= application.getServletContextName() %><br>
<% out.print("out.print() works.<br>"); out.println("out.println() works.<br>"); out.flush(); %>
```

Sample output: Request method: GET; Request URI: /YourProject/implicitMethods.jsp; Session ID: generated value; Application details and three out messages are displayed.

How to run: Run `implicitMethods.jsp`.

Viva question: What does `session.getId()` return?

Viva answer: The unique ID for the current session.

Q1-B - Question 29: Create a JSP application demonstrating Expression Language (EL).

Technology: JSP Expression Language (EL)

Where to write: `Web Pages/el.jsp`

Easy explanation: EL reads values without Java print statements. Here it displays studentName, marks and the web-app context path.

Code

```
<!-- File: el.jsp -->
<%@ page contentType="text/html;charset=UTF-8" %>
<%
    request.setAttribute("studentName", "Riya");
    request.setAttribute("marks", 85);
%>
<h2>Expression Language Demo</h2>
Student name: ${studentName}<br>
Marks: ${marks}<br>
Context path: ${pageContext.request.contextPath}
```

Sample output: Student name: Riya
Marks: 85
Context path: /YourProject

How to run: Save as `el.jsp` in Web Pages and run it on the server.

Viva question: What is JSP EL?

Viva answer: A simple way to access data in JSP using expressions such as `${studentName}`.

Q1-B - Question 30: Create a JSP application demonstrating JSTL.

Technology: JSP + JSTL

Where to write: `Web Pages/jstl.jsp`; add JSTL 1.2 library to Libraries

Easy explanation: JSTL provides tags like `c:forEach` and `c:out` to repeat and print values.

Code

```
<!-- File: jstl.jsp. Add JSTL 1.2 library to the project. -->
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ page import="java.util.*" %>
<%
    List<String> names = Arrays.asList("Riya", "Aman", "Sara");
    request.setAttribute("names", names);
%>
<h2>Students</h2>
<c:forEach var="n" items="${names}">
  <p><c:out value="${n}" /></p>
</c:forEach>
```

Sample output: Students
Riya
Aman
Sara

How to run: Add the JSTL library, then run `jstl.jsp`. If the tag library is missing, add the JSTL JAR to the project.

Viva question: What is JSTL?

Viva answer: JSP Standard Tag Library.

Q1-B - Question 31: Develop a JSP application demonstrating JSP Action Tags.

Technology: JSP Action Tags + JavaBean

Where to write: `Web Pages/action.jsp`, `Web Pages/footer.jsp`, `src/java/app/StudentBean.java`

Easy explanation: The JSP action tags create a bean, set and get its property, and include another JSP page.

Code

```
<!-- File: action.jsp -->
<jsp:useBean id="student" class="app.StudentBean" scope="request" />
<jsp:setProperty name="student" property="name" value="Riya" />
<h2>Action Tag Demo</h2>
Student name: <jsp:getProperty name="student" property="name" />
<hr>
<jsp:include page="footer.jsp" />

<!-- File: app/StudentBean.java -->
package app;
public class StudentBean {
    private String name;
    public StudentBean() { }
    public String getName() { return name; }
    public void setName(String name) { this.name = name; }
}

<!-- File: footer.jsp -->
<p>End of page.</p>
```

Sample output: Student name: Riya
End of page.

How to run: Create the three files at the paths shown and run `action.jsp`.

Viva question: Name one JSP action tag.

Viva answer: `jsp:include`, `jsp:useBean`, `jsp:setProperty` or `jsp:getProperty`.

Q1-B - Question 32: Develop a JSP application that passes values from one page to another and validates name, age, hobbies, email and gender.

Technology: JSP form validation

Where to write: `Web Pages/profileForm.jsp` and `Web Pages/profileResult.jsp`

Easy explanation: The first JSP collects values. The second JSP checks important fields and displays them. For real applications, escape displayed input to avoid unsafe HTML.

Code

```
<!-- File: profileForm.jsp -->
<form action="profileResult.jsp" method="post">
  Name: <input name="name" required><br>
  Age: <input name="age" type="number" min="1" required><br>
  Hobbies: <input name="hobbies" required><br>
  Email: <input name="email" type="email" required><br>
  Gender: <select name="gender"><option>Female</option><option>Male</option><option>Other</option></select><br>
  <button>Submit</button>
</form>

<!-- File: profileResult.jsp -->
<%@ page contentType="text/html;charset=UTF-8" %>
<%
String name=request.getParameter("name");
String age=request.getParameter("age");
String hobbies=request.getParameter("hobbies");
String email=request.getParameter("email");
String gender=request.getParameter("gender");
if (name==null || name.trim().isEmpty() || email==null || !email.contains("@")
    || age==null || age.isEmpty() || hobbies==null || hobbies.trim().isEmpty()
    || gender==null || gender.trim().isEmpty()) {
    out.println("Please enter all details correctly.");
} else if (Integer.parseInt(age) < 1) {
    out.println("Age must be positive.");
} else {
%>
<h2>Profile received</h2>
Name: <%= name %><br>Age: <%= age %><br>Hobbies: <%= hobbies %><br>
Email: <%= email %><br>Gender: <%= gender %>
<% } %>
```

Sample output: Profile received
Name: Riya; Age: 14; Hobbies: Reading; Email: riya@example.com; Gender: Female

How to run: Open `profileForm.jsp`, fill in all fields and submit.

Viva question: What is the purpose of the required attribute?

Viva answer: It prevents the browser from submitting an empty field.

Q1-B - Question 33: Create a JSP page that accepts employee number, employee name and salary, then inserts the record into an emp database table.

Technology: JSP + JDBC

Where to write: `Web Pages/empInsert.jsp`; shared `app.DB.java`; MySQL table `emp`

Easy explanation: The JSP inserts an employee row using PreparedStatement. The emp table and database helper must exist first.

Code

```
<!-- File: empInsert.jsp. Create table emp(empno INT PRIMARY KEY, ename VARCHAR(80), salary DOUBLE). -->
<%@ page import="java.sql.*" %>
<% if ("POST".equalsIgnoreCase(request.getMethod())) {
    try (Connection c=app.DB.getConnection();
         PreparedStatement p=c.prepareStatement("INSERT INTO emp(empno,ename,salary) VALUES(?,?,?)")) {
        p.setInt(1,Integer.parseInt(request.getParameter("empno")));
        p.setString(2,request.getParameter("ename"));
        p.setDouble(3,Double.parseDouble(request.getParameter("salary")));
        p.executeUpdate(); out.println("Employee inserted");
    } catch(Exception e) { out.println("Error: "+e.getMessage()); }
} %>
<form method="post">Employee No: <input name="empno" type="number"><br>
Name: <input name="ename"><br>Salary: <input name="salary" type="number" step="0.01"><br>
<button>Save</button></form>
```

Sample output: Employee inserted

How to run: Create the table, configure `app.DB`, and open `empInsert.jsp`.

Viva question: Which SQL command inserts a row?

Viva answer: `INSERT INTO`.

Q1-B - Question 34: Create an HTML form with employee number, name, age, designation and salary; submit the data to a JSP page that updates the matching employee record.

Technology: HTML + JSP + JDBC

Where to write: `Web Pages/empUpdate.html` and `Web Pages/empUpdate.jsp`; emp table must contain `age` and `designation`

Easy explanation: The JSP runs an UPDATE query using employee number as the matching key.

Code

```
<!-- File: empUpdate.html -->
<form action="empUpdate.jsp" method="post">
  Employee No: <input name="empno" type="number" required><br>
  Name: <input name="ename" required><br>
  Age: <input name="age" type="number" required><br>
  Designation: <input name="designation" required><br>
  Salary: <input name="salary" type="number" step="0.01" required><br><button>Update</button>
</form>

<!-- File: empUpdate.jsp. Add age and designation columns to emp table. -->
<%@ page import="java.sql.*" %>
<%
try (Connection c=app.DB.getConnection();
     PreparedStatement p=c.prepareStatement("UPDATE emp SET ename=?, age=?, designation=?, salary=? WHERE empno=?")) {
    p.setString(1,request.getParameter("ename")); p.setInt(2,Integer.parseInt(request.getParameter("age")));
    p.setString(3,request.getParameter("designation")); p.setDouble(4,Double.parseDouble(request.getParameter("salary")));
    p.setInt(5,Integer.parseInt(request.getParameter("empno")));
    int rows=p.executeUpdate(); out.println(rows==1 ? "Employee updated" : "Employee number not found");
} catch(Exception e) { out.println("Error: "+e.getMessage()); }
%>
```

Sample output: Employee updated

How to run: Add the required columns to `emp`, create the DB helper, and open `empUpdate.html`.

Viva question: What does UPDATE do?

Viva answer: It changes data in existing database rows.

Q1-B - Question 35: Develop a JSP application to authenticate a user's login against a database.

Technology: JSP + JDBC

Where to write: `Web Pages/dbLogin.jsp`; MySQL table `users`

Easy explanation: The JSP searches for a matching username and password. Use hashed passwords in real applications; this simple form is only for learning.

Code

```
<!-- File: dbLogin.jsp. Create users(username,password) and insert a test user. -->
<%@ page import="java.sql.*" %>
<% if ("POST".equalsIgnoreCase(request.getMethod())) {
    try (Connection c=app.DB.getConnection();
         PreparedStatement p=c.prepareStatement("SELECT username FROM users WHERE username=? AND password=?")) {
        p.setString(1,request.getParameter("username")); p.setString(2,request.getParameter("password"));
        try (ResultSet r=p.executeQuery()) {
            if (r.next()) out.println("Login successful. Hello " + r.getString(1));
            else out.println("Login failed");
        }
    } catch(Exception e) { out.println("Database error: "+e.getMessage()); }
} %>
<form method="post">Username: <input name="username"><br>Password: <input name="password" type="password"><br><button>Login</button></form>
```

Sample output: Login successful. Hello student

How to run: Create the users table and insert a test row, then open `dbLogin.jsp`.

Viva question: Why should passwords not be stored as plain text?

Viva answer: If the database leaks, plain-text passwords can be read directly.

Q1-B - Question 36: Create a JSP and JDBC gym membership portal that allows users to register and log in using database records.

Technology: JSP + JDBC

Where to write: `Web Pages/gymRegister.jsp`, `Web Pages/gymLogin.jsp`; table `gym(username,password)`

Easy explanation: One JSP adds a member to the database and another checks whether the credentials match a saved row.

Code

```
<!-- File: gymRegister.jsp -->
<%@ page import="java.sql.*" %>
<% if ("POST".equalsIgnoreCase(request.getMethod())) {
  try(Connection c=app.DB.getConnection(); PreparedStatement p=c.prepareStatement("INSERT INTO gym(username,password) VALUES(?,?)")) {
    p.setString(1,request.getParameter("username")); p.setString(2,request.getParameter("password"));
    p.executeUpdate(); out.println("Gym registration complete");
  } catch(Exception e) { out.println("Error: "+e.getMessage()); }
} %>
<form method="post">Username: <input name="username"><br>Password: <input name="password" type="password"><button>Register</button></form>

<!-- File: gymLogin.jsp -->
<%@ page import="java.sql.*" %>
<% if ("POST".equalsIgnoreCase(request.getMethod())) {
  try(Connection c=app.DB.getConnection(); PreparedStatement p=c.prepareStatement("SELECT username FROM gym WHERE username=? AND password=?
    p.setString(1,request.getParameter("username")); p.setString(2,request.getParameter("password"));
    try(ResultSet r=p.executeQuery()) { out.println(r.next()?"Login successful":"Login failed"); }
  } catch(Exception e) { out.println("Error: "+e.getMessage()); }
} %>
<form method="post">Username: <input name="username"><br>Password: <input name="password" type="password"><button>Login</button></form>
```

Sample output: Gym registration complete
Login successful

How to run: Create table `gym`, configure `app.DB`, register a test member, then log in.

Viva question: What does JDBC connect to?

Viva answer: A database from a Java application.

Q1-B - Question 37: Develop a JSP-based event registration form that accepts name, email, age and contact number, validates the input and displays the submitted details.

Technology: JSP

Where to write: `Web Pages/event.jsp`

Easy explanation: The JSP validates the name, email and contact number, then prints the submitted values.

Code

```
<!-- File: event.jsp -->
<%@ page contentType="text/html;charset=UTF-8" %>
<% if ("POST".equalsIgnoreCase(request.getMethod())) {
    String name=request.getParameter("name"), email=request.getParameter("email");
    String age=request.getParameter("age"), phone=request.getParameter("phone");
    if (name==null || name.trim().isEmpty() || email==null || !email.contains("@") || phone==null || phone.length()<10) {
        out.println("Please enter a name, valid email and 10-digit contact number.");
    } else {
        out.println("Registration details received:<br>Name: "+name+"<br>Email: "+email+"<br>Age: "+age+"<br>Contact: "+phone);
    }
} %>
<form method="post">Name: <input name="name" required><br>Email: <input name="email" type="email" required><br>
Age: <input name="age" type="number" min="1" required><br>Contact: <input name="phone" required><br><button>Register</button></form>
```

Sample output: Registration details received: Name: Riya; Email: riya@example.com; Age: 18; Contact: 9876543210

How to run: Open `event.jsp`, enter details and submit.

Viva question: What is server-side validation?

Viva answer: Validation performed by the server after receiving submitted data.

Q1-B - Question 38: Create a registration and login JSP application to register users and authenticate their credentials.

Technology: JSP + JDBC

Where to write: `Web Pages/register.jsp`, `Web Pages/login.jsp`; reuse `users` table

Easy explanation: Registration inserts a row; login checks for a matching row. This classroom example stores a plain password and should not be used for a real website.

Code

```
<!-- File: register.jsp -->
<%@ page import="java.sql.*" %>
<% if("POST".equalsIgnoreCase(request.getMethod())) {
  try(Connection c=app.DB.getConnection(); PreparedStatement p=c.prepareStatement("INSERT INTO users(username,password,email,country) VALUE
    p.setString(1,request.getParameter("username")); p.setString(2,request.getParameter("password"));
    p.setString(3,request.getParameter("email")); p.setString(4,request.getParameter("country")); p.executeUpdate();
    out.println("Registered successfully");
  } catch(Exception e) { out.println("Error: "+e.getMessage()); }
} %>
<form method="post">Username <input name="username"><br>Password <input type="password" name="password"><br>Email <input name="email"><br>C

<!-- File: login.jsp -->
<%@ page import="java.sql.*" %>
<% if("POST".equalsIgnoreCase(request.getMethod())) {
  try(Connection c=app.DB.getConnection(); PreparedStatement p=c.prepareStatement("SELECT username FROM users WHERE username=? AND password
    p.setString(1,request.getParameter("username")); p.setString(2,request.getParameter("password"));
    try(ResultSet r=p.executeQuery()) { out.println(r.next()?"Welcome "+r.getString(1):"Login failed"); }
  } catch(Exception e) { out.println("Error: "+e.getMessage()); }
} %>
<form method="post">Username <input name="username"><br>Password <input type="password" name="password"><br><button>Login</button></form>
```

Sample output: Registered successfully
Welcome student

How to run: Create the users table, register first, then open login.jsp.

Viva question: What is the difference between registration and login?

Viva answer: Registration creates an account; login checks an existing account.

Q1-B - Question 39: Develop a five-page web application using any two or three Java EE technologies.

Technology: Servlet + JSP + Session

Where to write: Create the JSP pages in Web Pages and servlet classes in src/java.

Easy explanation: This mini portal uses JSP for pages, Servlets for login/logout processing and a session to remember the name.

Code

```
<!-- Page 1: index.jsp -->
<h2>Student Portal</h2><a href="login.jsp">Login</a>

<!-- Page 2: login.jsp -->
<form action="login" method="post">Name: <input name="name"><button>Login</button></form>

// Page 3: LoginServlet.java
import java.io.*; import javax.servlet.annotation.WebServlet; import javax.servlet.http.*;
@WebServlet("/login") public class LoginServlet extends HttpServlet {
 protected void doPost(HttpServletRequest req,HttpServletResponse res)throws IOException {
  req.getSession().setAttribute("name",req.getParameter("name")); res.sendRedirect("welcome.jsp");
 }
}

<!-- Page 4: welcome.jsp -->
<h2>Hello ${sessionScope.name}</h2><a href="profile.jsp">Profile</a> | <a href="logout">Logout</a>

<!-- Page 5: profile.jsp -->
<h2>Profile page</h2>Name: ${sessionScope.name}
<a href="about.jsp">About</a>

<!-- Also create about.jsp as the fifth visible content page. -->
<h2>About the Student Portal</h2><p>Simple JSP and Servlet example.</p>

// LogoutServlet.java (extra controller class)
import java.io.*; import javax.servlet.annotation.WebServlet; import javax.servlet.http.*;
@WebServlet("/logout") public class LogoutServlet extends HttpServlet {
 protected void doGet(HttpServletRequest req,HttpServletResponse res)throws IOException {
  req.getSession().invalidate(); res.sendRedirect("index.jsp");
 }
}
```

Sample output: Pages: index.jsp, login.jsp, welcome.jsp, profile.jsp and about.jsp. Logout returns to index.jsp.

How to run: Run `index.jsp`, submit a name on login.jsp, visit profile.jsp and test Logout.

Viva question: Name two Java EE web technologies used here.

Viva answer: Servlets and JSP.

Q1-B - Question 40: Develop a Servlet application for a bakery that accepts cake quantity and price and calculates the total bill.

Technology: Servlet

Where to write: src/java/BakeryServlet.java

Easy explanation: The bill equals quantity multiplied by price per cake.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/bakery")
public class BakeryServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Cake quantity: <input type='number' name='qty' required></label><br>");
        out.println("<label>Price per cake: <input type='number' name='price' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        int qty=Integer.parseInt(request.getParameter("qty"));
```

```
        double price=Double.parseDouble(request.getParameter("price"));
        if(qty<0 || price<0) out.println("Values cannot be negative");
        else out.println("Total bill = Rs. "+(qty*price));
    }
}
```

Sample output: Quantity 3, price 250 -> Total bill = Rs. 750.0

How to run: Open `/bakery`, enter quantity and unit price.

Viva question: How is the total bill calculated?

Viva answer: Quantity multiplied by price per item.

Q1-B - Question 41: Create a Servlet application for a mobile shop that accepts mobile price and discount percentage and calculates the final price.

Technology: Servlet

Where to write: src/java/MobileDiscountServlet.java

Easy explanation: The discount amount is price times discount percent divided by 100. The final price is original price minus discount.

Code

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
@WebServlet("/mobilediscount")
public class MobileDiscountServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h2>Simple Form</h2>");
        out.println("<form method='post'>");
        out.println("<label>Mobile price: <input type='number' name='price' required></label><br>");
        out.println("<label>Discount percent: <input type='number' name='discount' required></label><br>");
        out.println("<button type='submit'>Submit</button></form>");
    }

    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws IOException {
        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        double price=Double.parseDouble(request.getParameter("price"));
        double d=Double.parseDouble(request.getParameter("discount"));
        if(price<0 || d<0 || d>100) { out.println("Enter a discount from 0 to 100"); return; }
        double finalPrice=price-(price*d/100.0);
        out.println("Discount amount = Rs. "+(price*d/100.0)+"<br>Final price = Rs. "+finalPrice);
    }
}
```

Sample output: Price 20000, discount 10 -> Discount amount 2000; Final price 18000

How to run: Open `/mobilediscount` and enter the price and percentage.

Viva question: What is the formula for discount amount?

Viva answer: Price x discount percentage / 100.

Q1-B - Question 42: Develop a Servlet application for a hostel registration portal that stores student details in a session and displays them on a confirmation page.

Technology: Servlet + Session + JSP

Where to write: `src/java/HostelServlet.java` and `Web Pages/hostelConfirm.jsp`

Easy explanation: The servlet keeps the name and room number in a session and redirects to a confirmation JSP.

Code

```
// File: HostelServlet.java
import java.io.*; import javax.servlet.*; import javax.servlet.annotation.WebServlet; import javax.servlet.http.*;
@WebServlet("/hostel") public class HostelServlet extends HttpServlet {
 protected void doGet(HttpServletRequest q,HttpServletResponse r)throws IOException {
  r.setContentType("text/html"); r.getWriter().println("<form method='post'>Name <input name='name'> Room <input name='room'><button>Regist
 }
 protected void doPost(HttpServletRequest q,HttpServletResponse r)throws IOException {
  HttpSession s=q.getSession(); s.setAttribute("hostelName",q.getParameter("name")); s.setAttribute("room",q.getParameter("room"));
  r.sendRedirect("hostelConfirm.jsp");
 }
}
```

Sample output: Hostel confirmation: Student Riya, Room 12

How to run: Create hostelConfirm.jsp with `${sessionScope.hostelName}` and `${sessionScope.room}`. Open `/hostel`.

Viva question: Why use a session for registration details?

Viva answer: So the details can be read on another page during the same visit.

Q1-B - Question 43: Create a JSP application using EL to display flight booking details, including passenger name, flight number, destination and ticket price.

Technology: JSP EL

Where to write: `Web Pages/flightForm.html` and `Web Pages/flight.jsp`

Easy explanation: EL reads values sent by the form using `${param.fieldName}`.

Code

```
<!-- File: flightForm.html -->
<form action="flight.jsp">Passenger <input name="passenger"><br>Flight no. <input name="flight"><br>
Destination <input name="destination"><br>Ticket price <input name="price"><br><button>Show</button></form>

<!-- File: flight.jsp -->
<h2>Flight Booking</h2>
Passenger: ${param.passenger}<br>Flight number: ${param.flight}<br>
Destination: ${param.destination}<br>Ticket price: Rs. ${param.price}
```

Sample output: Passenger: Riya; Flight number: AI101; Destination: Delhi; Ticket price: Rs. 4500

How to run: Open flightForm.html, enter values and submit.

Viva question: How does EL read a URL parameter?

Viva answer: Use `${param.nameOfParameter}`.

Q1-B - Question 44: Create a JSP application using EL to display student examination details, including roll number, name, subject and marks.

Technology: JSP EL

Where to write: `Web Pages/examForm.html` and `Web Pages/examDetails.jsp`

Easy explanation: EL displays values from the submitted form without writing Java code in the result page.

Code

```
<!-- File: examForm.html -->
<form action="examDetails.jsp">Roll no. <input name="roll"><br>Name <input name="name"><br>
Subject <input name="subject"><br>Marks <input name="marks"><br><button>Show</button></form>

<!-- File: examDetails.jsp -->
<h2>Exam Details</h2>
Roll number: ${param.roll}<br>Name: ${param.name}<br>Subject: ${param.subject}<br>Marks: ${param.marks}
```

Sample output: Roll number: 12; Name: Riya; Subject: Science; Marks: 19

How to run: Open examForm.html, enter data and submit.

Viva question: What does `${param.marks}` display?

Viva answer: The value submitted for the marks parameter.

Q1-B - Question 45: Create a JSP application using EL to display employee details received from another page, including employee ID, name, department and designation.

Technology: JSP EL

Where to write: `Web Pages/employeeForm.html` and `Web Pages/employeeDetails.jsp`

Easy explanation: The form sends employee data to another JSP, where EL prints each parameter.

Code

```
<!-- File: employeeForm.html -->
<form action="employeeDetails.jsp">ID <input name="id"><br>Name <input name="name"><br>
Department <input name="department"><br>Designation <input name="designation"><br><button>Show</button></form>

<!-- File: employeeDetails.jsp -->
<h2>Employee Details</h2>
ID: ${param.id}<br>Name: ${param.name}<br>Department: ${param.department}<br>Designation: ${param.designation}
```

Sample output: ID: 101; Name: Aman; Department: IT; Designation: Trainee

How to run: Open employeeForm.html, fill in the fields and submit.

Viva question: Why is EL simpler than scriptlets?

Viva answer: It displays values using a simple expression instead of Java code.

Q1-B - Question 46: Create a JSP application using EL to display product details received from another page, including product ID, name, quantity and price.

Technology: JSP EL

Where to write: `Web Pages/productForm.html` and `Web Pages/productDetails.jsp`

Easy explanation: EL prints each product field sent by the first page.

Code

```
<!-- File: productForm.html -->
<form action="productDetails.jsp">Product ID <input name="id"><br>Name <input name="name"><br>
Quantity <input name="quantity"><br>Price <input name="price"><br><button>Show</button></form>

<!-- File: productDetails.jsp -->
<h2>Product Details</h2>
ID: ${param.id}<br>Name: ${param.name}<br>Quantity: ${param.quantity}<br>Price: Rs. ${param.price}
```

Sample output: ID: P01; Name: Pen; Quantity: 10; Price: Rs. 5

How to run: Open productForm.html, enter product details and submit.

Viva question: Can EL display request parameters?

Viva answer: Yes, with the `param` implicit map.

Q1-B - Question 47: Create a JSP application using JSTL to display a list of available books with titles, authors and prices.

Technology: JSP + JSTL

Where to write: `Web Pages/books.jsp`; add JSTL 1.2 library

Easy explanation: The JSP stores two sample books in a list and JSTL loops through them to display a table.

Code

```
<!-- File: books.jsp. Add JSTL 1.2 library. -->
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>
<%@ page import="java.util.*" %>
<%
List<Map<String,Object>> books = new ArrayList<>();
Map<String,Object> b1=new HashMap<>(); b1.put("title","Java Basics"); b1.put("author","A. Kumar"); b1.put("price",250);
Map<String,Object> b2=new HashMap<>(); b2.put("title","Web Design"); b2.put("author","S. Rao"); b2.put("price",300);
books.add(b1); books.add(b2); request.setAttribute("books",books);
%>
<h2>Available Books</h2>
<table border="1"><tr><th>Title</th><th>Author</th><th>Price</th></tr>
<c:forEach var="b" items="${books}">
<tr><td><c:out value="${b.title}"/></td><td><c:out value="${b.author}"/></td><td><c:out value="${b.price}"/></td></tr>
</c:forEach></table>
```

Sample output: Java Basics | A. Kumar | 250
Web Design | S. Rao | 300

How to run: Run `books.jsp`.

Viva question: Which JSTL tag repeats items?

Viva answer: `c:forEach`.

Q2-A | EJB APPLICATIONS

Q2-A - Question 1: Create a currency converter using EJB to convert Indian Rupees into US Dollars, Euros and British Pounds.

Technology: EJB Stateless Session Bean

Where to write: Create `CurrencyBean.java` in a Java EE project. Deploy the project on Payara/GlassFish, not plain Tomcat.

Easy explanation: A stateless bean contains a conversion method. The exchange values are fixed examples for demonstrating the code, not current rates.

Code

```
import javax.ejb.Stateless;
@Stateless
public class CurrencyBean {
    // Example rates only. Update them before real use.
    public double convert(double inr, String currency) {
        switch (currency.toUpperCase()) {
            case "USD": return inr / 83.0;
            case "EUR": return inr / 90.0;
            case "GBP": return inr / 105.0;
            default: throw new IllegalArgumentException("Use USD, EUR or GBP");
        }
    }
}
// Example call from a Servlet:
// @EJB CurrencyBean converter;
// double dollars = converter.convert(8300, "USD");
```

Sample output: 8300 INR -> 100.0 USD (using the sample rate 83 INR per USD)

How to run: Deploy on Payara/GlassFish and inject with `@EJB CurrencyBean converter;` in a servlet or another bean.

Viva question: What is an EJB?

Viva answer: Enterprise JavaBeans are server-side components used for business logic.

Q2-A - Question 2: Develop a visitor hit-count application using a Singleton Session Bean.

Technology: EJB Singleton

Where to write: `src/java/HitCounterBean.java`

Easy explanation: A singleton bean has one shared instance for the application, so its counter is shared among visitors.

Code

```
import javax.ejb.Singleton;
import javax.ejb.Startup;
import javax.ejb.Lock;
import javax.ejb.LockType;
@Singleton
@Startup
public class HitCounterBean {
    private int count = 0;
    @Lock(LockType.WRITE)
    public int visit() { return ++count; }
    public int getCount() { return count; }
}
// In a Servlet: @EJB HitCounterBean counter;
// Print: "Visitor hits = " + counter.visit();
```

Sample output: Visitor hits = 1
Next request: Visitor hits = 2

How to run: Deploy on Payara/GlassFish. Inject it using `@EJB` into a servlet and call `visit()` on each request.

Viva question: Why use @Singleton?

Viva answer: To share one bean instance across the application.

Q2-A - Question 3: Develop a visitor statistics application using a Message-Driven Bean and Stateless Session Bean.

Technology: EJB MDB + Stateless Bean

Where to write: Create `VisitorStatsBean.java` and `VisitorMessageBean.java`; configure a JMS queue named `jms/VisitorQueue` in Payara/GlassFish.

Easy explanation: The message-driven bean receives queue messages and calls a stateless bean to record the event. A JMS queue must be configured in the server first.

Code

```
// File: VisitorStatsBean.java
import javax.ejb.Stateless;
@Stateless
public class VisitorStatsBean {
    public void record(String message) {
        System.out.println("Visitor event recorded: " + message);
    }
}

// File: VisitorMessageBean.java
import javax.ejb.MessageDriven;
import javax.ejb.ActivationConfigProperty;
import javax.ejb.EJB;
import javax.jms.Message;
import javax.jms.MessageListener;
import javax.jms.TextMessage;
@MessageDriven(activationConfig={
 @ActivationConfigProperty(propertyName="destinationLookup", propertyValue="jms/VisitorQueue"),
 @ActivationConfigProperty(propertyName="destinationType", propertyValue="javax.jms.Queue")
})
public class VisitorMessageBean implements MessageListener {
    @EJB VisitorStatsBean stats;
    public void onMessage(Message message) {
        try { stats.record(((TextMessage)message).getText()); }
        catch (Exception e) { e.printStackTrace(); }
    }
}
```

Sample output: Server log: Visitor event recorded: New visitor

How to run: Deploy on Payara/GlassFish after creating the JMS queue; send a text message to that queue to test it.

Viva question: What does an MDB do?

Viva answer: It receives messages asynchronously, commonly from a JMS queue.

Q2-A - Question 4: Develop a Marks Entry application using EJB to accept student details and marks, store them in a database, and retrieve and display the records.

Technology: EJB + JPA

Where to write: `src/java/Mark.java` and `src/java/MarksBean.java`; configure `META-INF/persistence.xml` as explained in setup

Easy explanation: The entity represents a marks row. The EJB saves a Mark object and retrieves all saved marks.

Code

```
// File: Mark.java
import javax.persistence.*;
@Entity
public class Mark {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String studentName; private int marks;
    public Mark() { }
    public Mark(String n,int m){studentName=n;marks=m;}
    public Long getId(){return id;} public String getStudentName(){return studentName;}
    public int getMarks(){return marks;}
}

// File: MarksBean.java
import java.util.*; import javax.ejb.Stateless; import javax.persistence.*;
@Stateless
public class MarksBean {
    @PersistenceContext(unitName="AppPU") EntityManager em;
    public void add(String name,int marks){ em.persist(new Mark(name,marks)); }
    public List<Mark> all(){ return em.createQuery("SELECT m FROM Mark m",Mark.class).getResultList(); }
}
```

Sample output: Saved: Riya, 88
Retrieved record: Riya - 88

How to run: Deploy on Payara/GlassFish, configure `AppPU`, call `marksBean.add("Riya",88)` and print `marksBean.all()`.

Viva question: What does EntityManager do?

Viva answer: It saves, finds, updates and removes entity objects in the database.

Q2-A - Question 5: Develop a room reservation system using EJB.

Technology: EJB Stateless Bean

Where to write: `src/java/RoomReservationBean.java`

Easy explanation: The bean checks whether a room is already in the sample reservation set. This tiny demo keeps data in memory; a real system should use a database.

Code

```
import javax.ejb.Stateless;
import java.util.HashSet;
import java.util.Set;
@Stateless
public class RoomReservationBean {
    private static final Set<String> demoReservations = new HashSet<>();
    public String reserve(String room, String guest) {
        if (!demoReservations.add(room)) return "Room already reserved";
        return "Room " + room + " reserved for " + guest;
    }
}
```

Sample output: Room 101 reserved for Riya

How to run: Deploy on Payara/GlassFish and call `reserve("101","Riya")` from a servlet.

Viva question: What is a business method?

Viva answer: A method that performs the application’s main work.

Q2-A - Question 6: Develop a shopping cart application using a Stateful Session Bean.

Technology: EJB Stateful Bean

Where to write: `src/java/ShoppingCartBean.java`

Easy explanation: A stateful bean remembers its list of items for a particular client session.

Code

```
import javax.ejb.Stateful;
import java.util.*;
@Stateful
public class ShoppingCartBean {
    private List<String> items = new ArrayList<>();
    public void add(String item){ items.add(item); }
    public void remove(String item){ items.remove(item); }
    public List<String> getItems(){ return new ArrayList<>(items); }
    public int count(){ return items.size(); }
}
```

Sample output: Items: [Book, Pen]
Count: 2

How to run: Deploy on Payara/GlassFish and call add/remove/getItems from a client or servlet.

Viva question: What is a stateful bean?

Viva answer: A bean that keeps conversational state for one client.

Q2-A - Question 7: Develop an electricity bill calculator using EJB that calculates the bill from units consumed.

Technology: EJB Stateless Bean

Where to write: `src/java/ElectricityBillBean.java`

Easy explanation: The bean calculates a bill using example slab rates. Replace the sample rates with the rates given by your teacher or electricity provider.

Code

```
import javax.ejb.Stateless;
@Stateless
public class ElectricityBillBean {
    public double bill(int units) {
        if (units < 0) throw new IllegalArgumentException("Units cannot be negative");
        if (units <= 100) return units * 3.0;
        if (units <= 300) return 100*3.0 + (units-100)*5.0;
        return 100*3.0 + 200*5.0 + (units-300)*7.0;
    }
}
```

Sample output: Units 150 -> Rs. 550.0 (using the demo slab rates)

How to run: Deploy and call `electricityBillBean.bill(150)` from a servlet or client.

Viva question: What is the purpose of the bill() method?

Viva answer: It calculates the bill from the number of units.

Q2-A - Question 8: Develop a temperature converter using EJB to convert Celsius to Fahrenheit.

Technology: EJB Stateless Bean

Where to write: `src/java/TemperatureBean.java`

Easy explanation: The formula is Fahrenheit = Celsius x 9/5 + 32.

Code

```
import javax.ejb.Stateless;
@Stateless
public class TemperatureBean {
    public double toFahrenheit(double celsius) {
        return (celsius * 9.0 / 5.0) + 32.0;
    }
}
```

Sample output: 25 Celsius -> 77.0 Fahrenheit

How to run: Deploy and call `temperatureBean.toFahrenheit(25)` from a servlet or client.

Viva question: What is the Celsius-to-Fahrenheit formula?

Viva answer: F = (C x 9/5) + 32.

Q2-A - Question 9: Develop an EJB-based hotel room reservation application that accepts guest details, room type, check-in date and number of nights.

Technology: EJB Stateless Bean

Where to write: `src/java/HotelReservationBean.java`

Easy explanation: The bean accepts guest, room type, date and nights, validates simple values and returns a confirmation message.

Code

```
import javax.ejb.Stateless;
@Stateless
public class HotelReservationBean {
    public String reserve(String guest, String roomType, String date, int nights) {
        if (guest == null || guest.trim().isEmpty() || nights < 1)
            return "Enter a guest name and at least one night";
        return "Reservation confirmed for " + guest + ", " + roomType
            + " room, check-in " + date + ", nights: " + nights;
    }
}
```

Sample output: Reservation confirmed for Riya, Deluxe room, check-in 2026-11-01, nights: 2

How to run: Deploy on Payara/GlassFish and call the method with four values from a servlet.

Viva question: Why validate number of nights?

Viva answer: To reject invalid values such as zero or negative nights.

Q2-A - Question 10: Develop an EJB-based shopping bill calculator that accepts product price and quantity and calculates the total amount.

Technology: EJB Stateless Bean

Where to write: `src/java/ShoppingBillBean.java`

Easy explanation: The total amount is unit price multiplied by quantity.

Code

```
import javax.ejb.Stateless;
@Stateless
public class ShoppingBillBean {
    public double total(double price, int quantity) {
        if (price < 0 || quantity < 0) throw new IllegalArgumentException("Values cannot be negative");
        return price * quantity;
    }
}
```

Sample output: Price 50, quantity 4 -> Total = 200.0

How to run: Deploy and call `shoppingBillBean.total(50,4)`.

Viva question: How do you calculate a shopping total?

Viva answer: Price multiplied by quantity.

Q2-A - Question 11: Develop an EJB-based salary calculator that accepts basic salary and allowances and calculates gross salary.

Technology: EJB Stateless Bean

Where to write: `src/java/SalaryBean.java`

Easy explanation: Gross salary is the basic salary plus allowances before deductions.

Code

```
import javax.ejb.Stateless;
@Stateless
public class SalaryBean {
    public double grossSalary(double basic, double allowances) {
        if (basic < 0 || allowances < 0) throw new IllegalArgumentException("Salary cannot be negative");
        return basic + allowances;
    }
}
```

Sample output: Basic = 30000, allowances = 5000 -> Gross salary = 35000.0

How to run: Deploy and call `salaryBean.grossSalary(30000,5000)`.

Viva question: What is gross salary?

Viva answer: Basic salary plus allowances before deductions.

Q2-A - Question 12: Create a Stateful Session Bean for a restaurant ordering system that allows customers to add and remove food items and calculate the total bill.

Technology: EJB Stateful Bean

Where to write: `src/java/RestaurantOrderBean.java`

Easy explanation: The stateful bean remembers each customer's order, can remove an item and totals the prices.

Code

```
import javax.ejb.Stateful;
import java.util.*;
@Stateful
public class RestaurantOrderBean {
    private Map<String,Double> order = new LinkedHashMap<>();
    public void add(String item,double price){ order.put(item,price); }
    public void remove(String item){ order.remove(item); }
    public double total(){ double sum=0; for(double p:order.values()) sum+=p; return sum; }
    public Set<String> items(){ return order.keySet(); }
}
```

Sample output: Add Pizza=200 and Juice=60 -> Total = 260.0; remove Juice -> Total = 200.0

How to run: Deploy and call `add`, `remove` and `total` from a servlet or client.

Viva question: Why use Stateful Session Bean here?

Viva answer: Because each customer's order must be remembered across several actions.

Q2-B | JPA APPLICATIONS

Q2-B - Question 13: Create a JPA application to store and retrieve book details.

Technology: JPA Entity + EJB service

Where to write: `src/java/Book.java` and `src/java/BookBean.java`; configure `AppPU`

Easy explanation: Book title, author and price are stored as columns. The bean saves and lists books.

Code

```
// File: Book.java
import javax.persistence.*;
@Entity
public class Book {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String title;
    private String author;
    private double price;
    public Book() { }
    public Book(String t,String a,double p) { title=t; author=a; price=p; }
    public Long getId(){return id;}
    public String getTitle(){return title;}
    public String getAuthor(){return author;}
    public double getPrice(){return price;}
}

// File: BookBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class BookBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String t,String a,double p){ em.persist(new Book(t,a,p)); }
    public List<Book> all(){ return em.createQuery("SELECT b FROM Book b",Book.class).getResultList(); }
}
```

Sample output: Saved: Java Basics, A. Kumar, 250; Retrieved book list contains the saved row.

How to run: Create `Book.java` and `BookBean.java` in `src/java`; configure `AppPU`; call `save()` and `all()` from a servlet or EJB client.

Viva question: What is an entity?

Viva answer: A Java class mapped to a database table.

Q2-B - Question 14: Create a JPA inventory application.

Technology: JPA Entity + EJB service

Where to write: `src/java/InventoryItem.java` and `src/java/InventoryItemBean.java`; configure `AppPU`

Easy explanation: The entity represents one item in stock. JPA saves and retrieves inventory records.

Code

```
// File: InventoryItem.java
import javax.persistence.*;
@Entity
public class InventoryItem {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String name;
    private int quantity;
    private double price;
    public InventoryItem() { }
    public InventoryItem(String n,int q,double p) { name=n; quantity=q; price=p; }
    public Long getId(){return id;}
    public String getName(){return name;}
    public int getQuantity(){return quantity;}
    public double getPrice(){return price;}
}

// File: InventoryItemBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class InventoryItemBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String n,int q,double p){ em.persist(new InventoryItem(n,q,p)); }
    public List<InventoryItem> all(){ return em.createQuery("SELECT i FROM InventoryItem i",InventoryItem.class).getResultList(); }
```

```
}
```

Sample output: Saved product: Notebook, quantity 20, price 40.0.

How to run: Create `InventoryItem.java` and `InventoryItemBean.java`; configure `AppPU`, then call `save()` and `all()`.

Viva question: What is inventory?

Viva answer: The list of products or goods a shop has in stock.

Q2-B - Question 15: Create a JPA guestbook application to store and retrieve visitor details and comments.

Technology: JPA Entity + EJB service

Where to write: `src/java/GuestComment.java` and `src/java/GuestCommentBean.java`; configure `AppPU`

Easy explanation: Each guestbook row stores a visitor name and comment.

Code

```
// File: GuestComment.java
import javax.persistence.*;
@Entity
public class GuestComment {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String visitorName;
    private String comment;
    public GuestComment() { }
    public GuestComment(String n,String c) { visitorName=n; comment=c; }
    public Long getId(){return id;}
    public String getVisitorName(){return visitorName;}
    public String getComment(){return comment;}
}

// File: GuestCommentBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class GuestCommentBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void add(String n,String c){ em.persist(new GuestComment(n,c)); }
    public List<GuestComment> all(){ return em.createQuery("SELECT g FROM GuestComment g",GuestComment.class).getResultList(); }
}
```

Sample output: Saved: Riya - Nice website!; retrieved in the guestbook list.

How to run: Create `GuestComment.java` and `GuestCommentBean.java`; configure `AppPU`; call `add()` and `all()`.

Viva question: What is a guestbook?

Viva answer: A page where visitors leave their names and comments.

Q2-B - Question 16: Create a JPA application demonstrating ORM associations.

Technology: JPA ORM association

Where to write: Create `Author.java` and `BookTitle.java`; add a service bean with `@PersistenceContext` to save `Author`.

Easy explanation: One author can have many books (`@OneToMany`). Each book points back to one author (`@ManyToOne`).

Code

```
// File: Author.java
import java.util.*; import javax.persistence.*;
@Entity public class Author {
 @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
 private String name;
 @OneToMany(mappedBy="author", cascade=CascadeType.ALL) private List<BookTitle> books=new ArrayList<>();
 public Author(){} public Author(String n){name=n;}
 public void addBook(BookTitle b){ books.add(b); b.setAuthor(this); }
 public String getName(){return name;} public List<BookTitle> getBooks(){return books;}
}
// File: BookTitle.java
import javax.persistence.*;
@Entity public class BookTitle {
 @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
 private String title;
 @ManyToOne private Author author;
 public BookTitle(){} public BookTitle(String t){title=t;}
 public void setAuthor(Author a){author=a;} public String getTitle(){return title;}
}
```

```
// File: AuthorBean.java
import javax.ejb.Stateless; import javax.persistence.*;
@Stateless public class AuthorBean {
 @PersistenceContext(unitName="AppPU") EntityManager em;
 public void saveDemo(){ Author a=new Author("A. Kumar"); a.addBook(new BookTitle("Java Basics")); em.persist(a); }
}
```

Sample output: Author: A. Kumar
Book: Java Basics

How to run: Configure `AppPU`, persist an Author with a BookTitle, and then query the author and its books.

Viva question: What is a One-to-Many relationship?

Viva answer: One row in one table can be related to several rows in another table.

Q2-B - Question 17: Create a JPA application to store employee names and salaries.

Technology: JPA Entity + EJB service

Where to write: `src/java/Employee.java` and `src/java/EmployeeBean.java`; configure `AppPU`

Easy explanation: JPA maps each Employee object to a row containing name and salary.

Code

```
// File: Employee.java
import javax.persistence.*;
@Entity
public class Employee {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String name;
    private double salary;
    public Employee() { }
    public Employee(String n,double s) { name=n; salary=s; }
    public Long getId(){return id;}
    public String getName(){return name;}
    public double getSalary(){return salary;}
}

// File: EmployeeBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class EmployeeBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String n,double s){ em.persist(new Employee(n,s)); }
    public List<Employee> all(){ return em.createQuery("SELECT e FROM Employee e",Employee.class).getResultList(); }
}
```

Sample output: Saved: Aman, salary 30000.0.

How to run: Create `Employee.java` and `EmployeeBean.java`; configure `AppPU`, then call save and all.

Viva question: What does @Entity mean?

Viva answer: It marks a class as a database entity.

Q2-B - Question 18: Create a JPA application to store course details and demonstrate ORM mapping.

Technology: JPA Entity + EJB service

Where to write: `src/java/Course.java` and `src/java/CourseBean.java`; configure `AppPU`

Easy explanation: ORM maps the Course Java object to a database table.

Code

```
// File: Course.java
import javax.persistence.*;
@Entity
public class Course {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String courseName;
    private int durationMonths;
    public Course() { }
    public Course(String n,int d) { courseName=n; durationMonths=d; }
    public Long getId(){return id;}
    public String getCourseName(){return courseName;}
    public int getDurationMonths(){return durationMonths;}
}
// File: CourseBean.java
```

```
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class CourseBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String n,int d){ em.persist(new Course(n,d)); }
    public List<Course> all(){ return em.createQuery("SELECT c FROM Course c",Course.class).getResultList(); }
}
```

Sample output: Course: Java Web, duration: 6 months.

How to run: Create `Course.java` and `CourseBean.java`; configure `AppPU`; call save and all.

Viva question: What is ORM?

Viva answer: Object-Relational Mapping; it maps Java objects to database tables.

Q2-B - Question 19: Create a JPA application to store product names and quantities.

Technology: JPA Entity + EJB service

Where to write: `src/java/Product.java` and `src/java/ProductBean.java`; configure `AppPU`

Easy explanation: The Product entity stores a product name and available quantity.

Code

```
// File: Product.java
import javax.persistence.*;
@Entity
public class Product {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String name;
    private int quantity;
    public Product() { }
    public Product(String n,int q) { name=n; quantity=q; }
    public Long getId(){return id;}
    public String getName(){return name;}
    public int getQuantity(){return quantity;}
}

// File: ProductBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class ProductBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String n,int q){ em.persist(new Product(n,q)); }
    public List<Product> all(){ return em.createQuery("SELECT p FROM Product p",Product.class).getResultList(); }
}
```

Sample output: Saved: Pen, quantity 100.

How to run: Create `Product.java` and `ProductBean.java`; configure `AppPU`, then call save and all.

Viva question: What is a primary key?

Viva answer: A field that uniquely identifies each database row.

Q2-B - Question 20: Create a JPA application demonstrating One-to-One mapping.

Technology: JPA One-to-One mapping

Where to write: Create `StudentProfile.java` and `StudentInfo.java`; configure `AppPU`.

Easy explanation: `@OneToOne` means one student record is linked to one profile record in this example.

Code

```
// File: StudentProfile.java
import javax.persistence.*;
@Entity public class StudentProfile {
 @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
 private String address;
 public StudentProfile(){} public StudentProfile(String a){address=a;}
 public String getAddress(){return address;}
}
// File: StudentInfo.java
import javax.persistence.*;
@Entity public class StudentInfo {
 @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
 private String name;
 @OneToOne(cascade=CascadeType.ALL) private StudentProfile profile;
 public StudentInfo(){} public StudentInfo(String n,StudentProfile p){name=n;profile=p;}
 public String getName(){return name;} public StudentProfile getProfile(){return profile;}
}
// File: StudentInfoBean.java
import javax.ejb.Stateless; import javax.persistence.*;
@Stateless public class StudentInfoBean {
 @PersistenceContext(unitName="AppPU") EntityManager em;
 public void saveDemo(){ em.persist(new StudentInfo("Riya", new StudentProfile("Mumbai"))); }
}
```

Sample output: Student: Riya
Address: Mumbai

How to run: Deploy on Payara/GlassFish and save a StudentInfo object with a StudentProfile object.

Viva question: What does @OneToOne mean?

Viva answer: It maps one entity to one related entity.

Q2-B - Question 21: Create a JPA application to store student names and roll numbers.

Technology: JPA Entity + EJB service

Where to write: `src/java/Student.java` and `src/java/StudentBean.java`; configure `AppPU`

Easy explanation: A Student entity maps name and roll number to a database row.

Code

```
// File: Student.java
import javax.persistence.*;
@Entity
public class Student {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String name;
    private int rollNo;
    public Student() { }
    public Student(String n,int r) { name=n; rollNo=r; }
    public Long getId(){return id;}
    public String getName(){return name;}
    public int getRollNo(){return rollNo;}
}

// File: StudentBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class StudentBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String n,int r){ em.persist(new Student(n,r)); }
    public List<Student> all(){ return em.createQuery("SELECT s FROM Student s",Student.class).getResultList(); }
}
```

Sample output: Saved student: Riya, roll number 12.

How to run: Create `Student.java` and `StudentBean.java`; configure `AppPU`, then call save and all.

Viva question: Why use a roll number?

Viva answer: It is used to identify a student; it should be unique in the class.

Q2-B - Question 22: Create a JPA application to store student names and marks.

Technology: JPA Entity + EJB service

Where to write: `src/java/StudentMarkRecord.java` and `src/java/StudentMarkRecordBean.java`; configure `AppPU`

Easy explanation: The entity stores a student's name and marks.

Code

```
// File: StudentMarkRecord.java
import javax.persistence.*;
@Entity
public class StudentMarkRecord {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String name;
    private int marks;
    public StudentMarkRecord() { }
    public StudentMarkRecord(String n,int m) { name=n; marks=m; }
    public Long getId(){return id;}
    public String getName(){return name;}
    public int getMarks(){return marks;}
}

// File: StudentMarkRecordBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class StudentMarkRecordBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String n,int m){ em.persist(new StudentMarkRecord(n,m)); }
    public List<StudentMarkRecord> all(){ return em.createQuery("SELECT s FROM StudentMarkRecord s",StudentMarkRecord.class).getResultList(
}
```

Sample output: Saved: Riya, marks 19.

How to run: Create `StudentMarkRecord.java` and `StudentMarkRecordBean.java`; configure `AppPU`; call save and all.

Viva question: What is a database record?

Viva answer: One row of stored information.

Q2-B - Question 23: Create a JPA application to store book names and prices.

Technology: JPA Entity + EJB service

Where to write: `src/java/PricedBook.java` and `src/java/PricedBookBean.java`; configure `AppPU`

Easy explanation: Each row saves a book name and its price.

Code

```
// File: PricedBook.java
import javax.persistence.*;
@Entity
public class PricedBook {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String name;
    private double price;
    public PricedBook() { }
    public PricedBook(String n,double p) { name=n; price=p; }
    public Long getId(){return id;}
    public String getName(){return name;}
    public double getPrice(){return price;}
}

// File: PricedBookBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class PricedBookBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String n,double p){ em.persist(new PricedBook(n,p)); }
    public List<PricedBook> all(){ return em.createQuery("SELECT b FROM PricedBook b",PricedBook.class).getResultList(); }
}
```

Sample output: Saved: Web Design, price 300.0.

How to run: Create `PricedBook.java` and `PricedBookBean.java`; configure `AppPU`; call save and all.

Viva question: How does JPA retrieve all books?

Viva answer: With a query such as `SELECT b FROM PricedBook b`.

Q2-B - Question 24: Create a JPA-based student management system to insert, display and update student details.

Technology: JPA CRUD

Where to write: `src/java/ManagedStudent.java` and `src/java/StudentManagerBean.java`

Easy explanation: The service inserts students, lists all students and updates a name when the roll number exists.

Code

```
// File: ManagedStudent.java
import javax.persistence.*;
@Entity public class ManagedStudent {
 @Id private int rollNo; private String name; private int age;
 public ManagedStudent(){} public ManagedStudent(int r,String n,int a){rollNo=r;name=n;age=a;}
 public int getRollNo(){return rollNo;} public String getName(){return name;} public int getAge(){return age;}
 public void setName(String n){name=n;} public void setAge(int a){age=a;}
}
// File: StudentManagerBean.java
import java.util.*; import javax.ejb.Stateless; import javax.persistence.*;
@Stateless public class StudentManagerBean {
 @PersistenceContext(unitName="AppPU") EntityManager em;
 public void add(int r,String n,int a){em.persist(new ManagedStudent(r,n,a));}
 public List<ManagedStudent> all(){return em.createQuery("SELECT s FROM ManagedStudent s",ManagedStudent.class).getResultList();}
 public void updateName(int r,String n){ManagedStudent s=em.find(ManagedStudent.class,r); if(s!=null)s.setName(n);}
}
```

Sample output: Inserted student 12, Riya, age 14
Updated name: Riya Shaikh

How to run: Configure `AppPU`; call `add`, `all` and `updateName` from a servlet or client.

Viva question: What does CRUD stand for?

Viva answer: Create, Read, Update and Delete.

Q2-B - Question 25: Create a JPA-based employee management system to insert, display and update employee details.

Technology: JPA CRUD

Where to write: `src/java/ManagedEmployee.java` and `src/java/EmployeeManagerBean.java`

Easy explanation: The service inserts employee data, displays all rows and updates salary by employee number.

Code

```
// File: ManagedEmployee.java
import javax.persistence.*;
@Entity public class ManagedEmployee {
 @Id private int empNo; private String name; private double salary;
 public ManagedEmployee(){} public ManagedEmployee(int e,String n,double s){empNo=e;name=n;salary=s;}
 public int getEmpNo(){return empNo;} public String getName(){return name;} public double getSalary(){return salary;}
 public void setName(String n){name=n;} public void setSalary(double s){salary=s;}
}
// File: EmployeeManagerBean.java
import java.util.*; import javax.ejb.Stateless; import javax.persistence.*;
@Stateless public class EmployeeManagerBean {
 @PersistenceContext(unitName="AppPU") EntityManager em;
 public void add(int e,String n,double s){em.persist(new ManagedEmployee(e,n,s));}
 public List<ManagedEmployee> all(){return em.createQuery("SELECT e FROM ManagedEmployee e",ManagedEmployee.class).getResultList();}
 public void updateSalary(int e,double s){ManagedEmployee x=em.find(ManagedEmployee.class,e);if(x!=null)x.setSalary(s);}
}
```

Sample output: Inserted employee 101, Aman, salary 30000
Updated salary: 35000

How to run: Configure `AppPU`; call `add`, `all` and `updateSalary`.

Viva question: Which field identifies an employee here?

Viva answer: `empNo`, the primary key.

Q2-B - Question 26: Create a JPA-based grocery store application to insert and display product ID, name, quantity and price.

Technology: JPA Entity + EJB service

Where to write: `src/java/GroceryProduct.java` and `src/java/GroceryProductBean.java`; configure `AppPU`

Easy explanation: The generated `id` is the product ID. The other fields store the name, quantity and price.

Code

```
// File: GroceryProduct.java
import javax.persistence.*;
@Entity
public class GroceryProduct {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String productName;
    private int quantity;
    private double price;
    public GroceryProduct() { }
    public GroceryProduct(String n,int q,double p) { productName=n; quantity=q; price=p; }
    public Long getId(){return id;}
    public String getProductName(){return productName;}
    public int getQuantity(){return quantity;}
    public double getPrice(){return price;}
}

// File: GroceryProductBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class GroceryProductBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String n,int q,double p){ em.persist(new GroceryProduct(n,q,p)); }
    public List<GroceryProduct> all(){ return em.createQuery("SELECT g FROM GroceryProduct g",GroceryProduct.class).getResultList(); }
}
```

Sample output: P01 (generated ID), Rice, 10, price 60.0.

How to run: Create `GroceryProduct.java` and `GroceryProductBean.java`; configure `AppPU`; call save and all.

Viva question: What does @GeneratedValue do?

Viva answer: It asks JPA to generate primary-key values automatically.

Q2-B - Question 27: Create a JPA-based student result system to store student roll number, name, subject and marks.

Technology: JPA Entity + Query

Where to write: `src/java/StudentResult.java` and a service bean using `@PersistenceContext`

Easy explanation: The entity stores roll number, student name, subject and marks. A JPQL query retrieves saved results.

Code

```
// File: StudentResult.java
import javax.persistence.*;
@Entity public class StudentResult {
 @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
 private int rollNo; private String name; private String subject; private int marks;
 public StudentResult(){}
 public StudentResult(int r,String n,String s,int m){rollNo=r;name=n;subject=s;marks=m;}
 public Long getId(){return id;} public int getRollNo(){return rollNo;} public String getName(){return name;}
 public String getSubject(){return subject;} public int getMarks(){return marks;}
}
// File: StudentResultBean.java
import java.util.*; import javax.ejb.Stateless; import javax.persistence.*;
@Stateless public class StudentResultBean {
 @PersistenceContext(unitName="AppPU") EntityManager em;
 public void add(int roll,String name,String subject,int marks){em.persist(new StudentResult(roll,name,subject,marks));}
 public List<StudentResult> all(){return em.createQuery("SELECT r FROM StudentResult r",StudentResult.class).getResultList();}
}
```

Sample output: Roll 12 | Riya | Science | 19

How to run: Configure `AppPU`, create a service bean, persist a StudentResult and run the SELECT query.

Viva question: What is JPQL?

Viva answer: Java Persistence Query Language; it queries entity objects.

Q2-B - Question 28: Create a JPA-based mobile shop application to insert, display and update mobile phone details.

Technology: JPA CRUD

Where to write: `src/java/MobilePhone.java` and `src/java/MobileShopBean.java`

Easy explanation: The model number is the primary key. The bean inserts, lists and updates phone prices.

Code

```
// File: MobilePhone.java
import javax.persistence.*;
@Entity public class MobilePhone {
 @Id private String modelNo; private String brand; private double price;
 public MobilePhone(){} public MobilePhone(String m,String b,double p){modelNo=m;brand=b;price=p;}
 public String getModelNo(){return modelNo;} public String getBrand(){return brand;} public double getPrice(){return price;}
 public void setPrice(double p){price=p;}
}
// File: MobileShopBean.java
import java.util.*; import javax.ejb.Stateless; import javax.persistence.*;
@Stateless public class MobileShopBean {
 @PersistenceContext(unitName="AppPU") EntityManager em;
 public void add(String m,String b,double p){em.persist(new MobilePhone(m,b,p));}
 public List<MobilePhone> all(){return em.createQuery("SELECT m FROM MobilePhone m",MobilePhone.class).getResultList();}
 public void updatePrice(String m,double p){MobilePhone x=em.find(MobilePhone.class,m);if(x!=null)x.setPrice(p);}
}
```

Sample output: Added: M01, ExampleBrand, Rs. 15000
Updated price: Rs. 14000

How to run: Configure `AppPU`; call add, all and updatePrice.

Viva question: What does em.find() do?

Viva answer: It searches for an entity using its primary key.

Q2-B - Question 29: Create a JPA-based vehicle showroom application to insert and display vehicle ID, model name and price.

Technology: JPA Entity + EJB service

Where to write: `src/java/Vehicle.java` and `src/java/VehicleBean.java`; configure `AppPU`

Easy explanation: The vehicle ID is generated, and model name and price are saved.

Code

```
// File: Vehicle.java
import javax.persistence.*;
@Entity
public class Vehicle {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String modelName;
    private double price;
    public Vehicle() { }
    public Vehicle(String m,double p) { modelName=m; price=p; }
    public Long getId(){return id;}
    public String getModelName(){return modelName;}
    public double getPrice(){return price;}
}

// File: VehicleBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class VehicleBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String m,double p){ em.persist(new Vehicle(m,p)); }
    public List<Vehicle> all(){ return em.createQuery("SELECT v FROM Vehicle v",Vehicle.class).getResultList(); }
}
```

Sample output: Vehicle ID 1, model City, price 1200000.0.

How to run: Create `Vehicle.java` and `VehicleBean.java`; configure `AppPU`; call save and all.

Viva question: What kind of values are stored in a vehicle entity?

Viva answer: Vehicle ID, model name and price.

Q2-B - Question 30: Create a JPA-based bookshop application to insert and display book details.

Technology: JPA Entity + EJB service

Where to write: `src/java/ShopBook.java` and `src/java/ShopBookBean.java`; configure `AppPU`

Easy explanation: The bookshop saves and lists title, author and price for each book.

Code

```
// File: ShopBook.java
import javax.persistence.*;
@Entity
public class ShopBook {
    @Id @GeneratedValue(strategy=GenerationType.AUTO) private Long id;
    private String title;
    private String author;
    private double price;
    public ShopBook() { }
    public ShopBook(String t,String a,double p) { title=t; author=a; price=p; }
    public Long getId(){return id;}
    public String getTitle(){return title;}
    public String getAuthor(){return author;}
    public double getPrice(){return price;}
}

// File: ShopBookBean.java
import java.util.*;
import javax.ejb.Stateless;
import javax.persistence.*;
@Stateless
public class ShopBookBean {
    @PersistenceContext(unitName="AppPU") private EntityManager em;
    public void save(String t,String a,double p){ em.persist(new ShopBook(t,a,p)); }
    public List<ShopBook> all(){ return em.createQuery("SELECT b FROM ShopBook b",ShopBook.class).getResultList(); }
}
```

Sample output: Java Basics, A. Kumar, 250.0.

How to run: Create `ShopBook.java` and `ShopBookBean.java`; configure `AppPU`; call save and all.

Viva question: What is the role of `persist()`?

Viva answer: It tells JPA to insert a new entity into the database.

Final Checklist Before Running

• For Servlet/JSP: confirm that Tomcat is started and the project builds without errors.

• If a servlet URL returns 404, check the @WebServlet path and project context name.

• If a database question fails, check MySQL is running, the database/table exists, the password is correct and Connector/J is added.

• For JSP JSTL questions, make sure the JSTL library is included.

• For EJB/JPA questions, deploy to Payara/GlassFish and check the configured JNDI data source and persistence-unit name.

• Use the sample output as a guide; generated IDs, server names and session IDs can be different on your computer.

• These snippets are beginner classroom examples. For real websites, use validation, safe output escaping, secure password hashing, authorization and proper error handling.

Official reference links for compatibility and concepts

Apache Tomcat 9 documentation (Servlet 4.0 / JSP 2.3)

Jakarta Persistence explanation (entities, EntityManager and persistence.xml)

Java EE 8 / javax.persistence API

Payara documentation about javax to jakarta namespace changes